End-to-end Encryption (E2EE) Secure Instant Messenger Project Report

Team19

Au Tsz Kin 23085232d, Cheung Kei Yau 23088621d, Lau Ming Yin 23087882d, Lee Yin Lam Elaine 23082545d

***Abstract*— We built a secure 1:1 instant messaging system with end-to-end encryption (E2EE) where message confidentiality and integrity are protected against an honest-but-curious (HbC) server. Clients establish authenticated shared secrets using long-term identity keys and authenticated key agreement, then encrypt each message using an AEAD scheme with authenticated associated data (AD). The system supports timed self-destruct messages, friend request workflows, offline store-and-forward delivery of ciphertext, per-conversation unread counters with incremental conversation loading, and delivery status reporting ("Sent" and "Delivered") with clearly defined semantics. Replay and duplicate ciphertext are detected and ignored using per-conversation sequencing and receiver-side de-duplication state. Password + OTP authentication is used for account access, while TLS protects client–server transport to mitigate network attackers and credential theft.**

I. Introduction

This report outlines the design, implementation, and security evaluation of a customized end-to-end encrypted (E2EE) instant messaging (IM) platform. The system is built under an Honest-but-Curious (HbC) server threat model, the system facilitates 1:1 private, single-device communication via a FastAPI WebSocket server and a lightweight Python client. The primary objective is to facilitate reliable communication between users while cryptographically guaranteeing that the server routing the traffic cannot read or alter the contents of the messages.

Beyond basic messaging, the application supports comprehensive user features, including user authentication, via password and one-time password (OTP), friend contact management, offline store-and-forward queueing, delivery receipts, and timed self-destructing (TTL) messages.

To defend against both active network adversaries and the HbC server, the architecture enforces strict cryptographic and operational boundaries. Plaintext confidentiality and integrity are maintained using AES-256-GCM (AEAD) applied client-side, with all transport encapsulated in TLS. The client monitors local message states to reject replay attacks and actively monitors identity keys via a Trust on First Use (TOFU) policy to avoid tampering and impersonation. Finally, the system minimizes metadata leakage and platform abuse by enforcing strict server-side bounds, including a 7-day maximum retention policy for offline queues and rate-limiting controls.

II. Threat Model & Assumptions

The security considerations of our IM platform is developed based on a threat model and underlying assumptions. We categorize these into active adversaries within the protocol and components that fall out of its security perimeter.

1. *Threat Model*

- *HbC Server:* The server is assumed to be compliant with the protocol that it routes messages and stores ciphertexts as intended, but may inspect databases, logs, and all application-layer data, and perform traffic analysis (e.g., timestamps, message sizes, and the contact graph). To reduce this threat, the platform assigns all cryptographic functions to the client side. The server ensures that it has no knowledge of the contents of plaintext messages. Furthermore, strict input validation (e.g., 10KB payload limits) prevents the server from easily deducing message context purely through size analysis of extreme payloads.
- *External Network Attacker*: It is assumed that there are network level adversaries capable of monitoring all communication channels between a client and a server. If transport security is misconfigured, this attacker may attempt MITM attacks in order to intercept or modify data in transit. Additionally, the attacker may manipulate the network layer to cause packet duplication or reordering.
- *Malicious Client:* A registered user may modify their local client code to bypass UI restrictions, attempting to spam other users or inject malformed data to crash the server. The server acts as a zero-trust gateway. Unauthorized spam scripts are eliminated by enforcing server-side database checks for friendship status prior to permitting WebSocket relays.

2. *Security Assumptions*

- *Endpoint Security*: It is assumed that the user's physical device (laptop/PC) is not compromised. If the endpoint device are having a keylogger or a rootkit installed, no

E2EE can protect the message content before it is encrypted or after it is decrypted.

- *Secure Randomness*: It is assumed that the operating system provides a cryptographically secure pseudo-random number generator (CSPRNG) for key generation and nonce creation.

3. Out-of-Scope Items

As defined in project scope, the following threats are explicitly acknowledged but considered out-of-scope for our defensive implementations:

- *Metadata Obfuscation*: Communication metadata cannot be protected from the server, even the message content is completely private. The HbC server automatically detects when users are online, with whom they are chatting, and how frequently they do so.
- *Analog/Endpoint Data Exfiltration*: Timed self-destructing messages is implemented in the application; however, this feature operates on a "best-effort" basis. It is acknowledged that self-destruct cannot prevent screenshots, copy/paste, or a malicious client.

III. ARCHITECTURE

The design implements a client-server architecture aimed at segregating the encryption activities from the central node. The design architecture is split into two main execution models, namely the Trusted Client Model and the Untrusted Server Model.

1. System Components

- *Client (Python)*: A terminal application that manages the user's permanent identity keys and temporary Prekeys. It performs all encryption, decryption, and key exchange.
- *Server (FastAPI/SQLite)*: A centralized controller that manages user accounts, authenticates sessions via JWT-like tokens, stores public key packages, and provides a "store-and-forward" queue for offline messages.
- *Communication Channels*: `*HTTPS` endpoints: For handling registration, login, friend requests, and fetching public key packages.
- *WSS (WebSockets)*: Used for stateful, full-duplex, real-time message delivery and instant status updates.

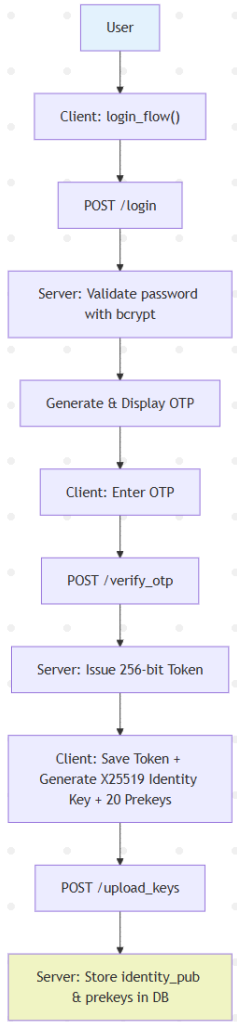
2. Trust Boundaries

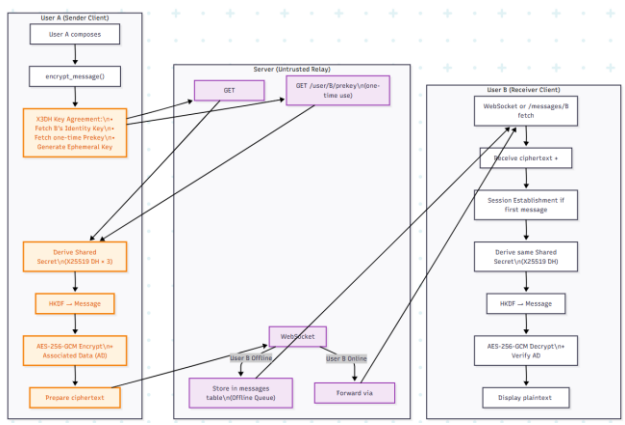
The primary trust boundary exists at the Network Interface of the client application.

At the client side, data is handled in plaintext. The X25519 keys of the client and session keys of AES-GCM encryption used by the client are within this boundary. We assume that there is no risk of memory scraping on the client side.

At the server and network side, when data leaves the client, it is considered to be in an untrusted zone. Even though TLS provides transport-layer security, the message content in application-layer is already encrypted, ensuring that even if the TLS tunnel is terminated at the server, the server administrator would sees only ciphertext.

3. Data Flow Diagrams





This is a data flow diagram showcasing the data flow between User A (sender), server, and User B (receiver).

After user A composes a message, it is encrypted by the `encrypt_message()` function. According to the X3DH Agreement, user B's identity key and OTP are fetched, as well as the ephemeral key. These are sent to the server to GET. The shared secret is then derived according to the X25519 DH*3.

HKDF is derived to message key, using the AES-256-GCM encryption adding the associated data. The ciphertext package is prepared on the user A's client. Depending on user B's online status, the message is forwarded via WebSocket if online, else stored in messages table, an offline queue. User B client fetches the message, receives ciphertext and header, and establishes a session if it is the first message between them. The same shared secret (X25519 DH) is derived, HKDF to message key. It is decrypted by AES-256-GCM and the associated data is verified. Finally, the plaintext message is displayed.

IV. PROTOCOL DESIGN

1. *Session Establishment (3DH Handshake)*

The system implements an asynchronous Extended Triple Diffie-Hellman (X3DH) handshake using X25519 and HKDF. This allows two users to establish a shared secret even if one is offline. The design uses three types of keys:

- *Identity key pair:* long-term key pair generated and stored locally by each client.
- *One-time prekey pair*: generated by the receiver and uploaded to the server for offline session initiation.
- *Ephemeral key pair*: freshly generated by the sender for each new session.

When a sender initiates a session with a contact, the client first fetches the contact's identity public key from the server and checks for changes to the previously stored identity key. If the identity key differs from the previously stored one, the session establishment is aborted and the user is warned. This gives visibility to key changes.

Next, the sender fetches one of the recipient's one-time prekeys from the server. The sender then computes three Diffie–Hellman values:

```
their_prekey_pub = x25519.X25519PublicKey.from_public_bytes(
    base64.b64decode(prekey_data["pub"])
)

# Derive long-term shared secret
dh1 = state.identity_priv.exchange(their_prekey_pub)
dh2 = ephemeral.exchange(their_id_pub)
dh3 = ephemeral.exchange(their_prekey_pub)
```

These values are concatenated and passed into HKDF to derive the initial shared root key:

`root_key = hkdf(dh1 + dh2 + dh3, "long-term-secret")`

The receiver derives the same root key using its own identity private key, the matching prekey private key, and the sender's ephemeral public key. After derivation, both sides store the resulting shared secret locally together with send/receive counters initialized to zero. This state is later used for encrypted messaging and replay protection.

2. *Message Formats*

To satisfy (R8), we use AES-256-GCM for authenticated encryption. Each message is encapsulated in a JSON structure containing a base64-encoded ciphertext, a unique twelve-byte initialization vector, and an ephemeral public key for the specific exchange. The system uses AES-256-GCM and binds pertinent metadata, namely the sender and recipient identities, as Authenticated Associated Data to guarantee integrity and thwart misdirection attacks. This binding ensures that the recipient's decryption will fail if the message headers or intent are tampered with.

3. *State Machine*

Each client maintains a local state machine (managed in `client_state.json`) to track the progression of these security properties over time. To preserve synchronization and offer the cryptographic basis for replay detection, the state contains a monotonic counter for both sent and received messages. By guaranteeing that any message coming with a counter value less than or equal to the previously recorded state is promptly recognized and rejected as a potential replay attack.

Upon successful key derivation, the client transitions into the Active Session phase; in this state, it manages a symmetric ratchet by tracking monotonic `counter_send` and `counter_recv` values for every contact to ensure that no encryption key is ever reused. When the server session expires after 24 hours, the `client.py` will have a local cleanup mechanism. Finally, the system enters the Exited or Logged Out state when the user terminates the session, which triggers the local clearing of session tokens and the closure of all active WebSocket connections to prevent unauthorized access to the application-layer state.

4. *Replay Handling and De-duplication*

The protocol implements a secondary layer of de-duplication. A local cache of observed message identifiers is kept to manage benign network retries while the cryptographic counters guard against malicious replays. When the server resends a message due to a connection drop, the client checks the database-level message ID against its local cache. If a match is found, the duplicate is silently ignored rather than triggering a security exception, ensuring a seamless user experience without compromising the underlying security model.

5. *Rate Limiting and Abuse Control*

To protect the integrity of the protocol against automated attacks and brute-force attempts, the system implements multi-layered rate limiting at the server boundary. This mechanism tracks the IP address of incoming requests and applies a cooldown period once a certain threshold of requests is exceeded within a window. By limiting these activities, the protocol protects against password-spraying attacks and prevents malicious users from exhausting the server's database resources by creating multiple accounts. Furthermore, rate limiting is applied to the OTP verification phase, ensuring that the code cannot be recovered through a brute-force search within its valid lifetime. If a client exceeds this threshold, the

server rejects the request, aborts database operations, and returns an HTTP 429 “Too Many Requests” status code. The response payload includes the exact integer of remaining time until the window resets, allowing honest clients to gracefully pause operations.

V. Cryptographic Choices & Rationale

The implementation prioritizes modern, high-assurance cryptographic primitives. We utilize the Python `cryptography` library (specifically the `hazmat` layer for low-level primitives), which offers strong encapsulation of OpenSSL.

1. Cryptographic Primitive Selection

Function	Primitive / Algorithm	Parameter / Specification
Key Agreement & Identity (X3DH-style)	X25519 (ECDH)	256-bit Curve25519
Symmetric Encryption	AES-256-GCM	256-bit key, 96-bit Nonce, 128-bit Tag
Local State Encryption	PBKDF2HMAC	SHA-256, 480,000 iterations
Key Derivation (KDF)	HKDF-SHA256	HMAC-based, 32-byte salt/info
Password Hashing	Bcrypt	Cost factor 12, CSPRNG-generated salt
Transport Security	TLS 1.3 / 1.2	Self-signed RSA-2048/4096 Certificates

2. Rationale by Objective

A. Confidentiality and Forward Secrecy

X25519 was chosen for the 3DH key exchange due to its high performance, small key size, and 128-bit security level. It enables asynchronous session establishment while guaranteeing session-level forward secrecy. Even if the long-term identity key is later compromised, past messages remain secure.

B. Integrity and Authenticity (AEAD)

AES-GCM guarantees both confidentiality and data integrity. The 128-bit authentication tag ensures that if the HbC server or a network attacker attempts to flip bits in the ciphertext will lead to failure during decryption, hence preventing the injection of malformed data.

C. Entropy and Key Hardening

The raw shared secrets obtained from an ECDH exchange may not be uniformly distributed. To get a high-entropy 256-bit AES key, we “extract and expand” our raw X25519 secret with the help of the HKDF-SHA256 algorithm. For local state protection, PBKDF2HMAC derives a strong encryption key from the user's local passphrase.

D. Identity and Replay Protection

The system employs a "Trust on First Use" (TOFU) model, alerting users if a peer's X25519 identity key changes. To prevent replay attacks, the client tracks UUIDv4 message IDs and local counters, ensuring duplicated ciphertexts are rejected.

3. Secure Randomness and Key Generation

Our system used `X25519PrivateKey.generate()` from `cryptography` library, `os.urandom` from `OS CSPRNG`, and `uuid.uuid4()` from Python `uuid` for generating randomness. Our system strictly uses cryptographically secure pseudo-random number generators (CSPRNGs). It completely avoids Python’s standard random module, which is unsuitable for security.

All secret materials — including identity keys, prekeys, ephemeral keys, nonces, salts, and session tokens — are generated exclusively using the operating system’s CSPRNG through `os.urandom()` and the `cryptography` library. This ensures high entropy by drawing hardware events, timing, and other unpredictable sources.

Our system correctly avoids manual seeding and never uses Python’s weak random module. Secure random generation is delegated to the well-audited cryptography library, which calls platform-specific primitives safely. Independent random values are generated for keys, nonces, salts, and tokens to maintain diversity and avoid single points of failure. uuid.uuid4() is used only for non-critical message IDs.

4. Technical Rationale for Library Choice

A. The `cryptography` library (Hazmat Layer)

The `hazmat` (Hazardous Materials) layer was selected for X25519, HKDF, AES-GCM, PBKDF2, and serialization primitives. Those can be used to ensure the application utilizes low-level, high-performance primitives. By mapping directly to audited OpenSSL calls, this approach provides strong memory security and resistance to common side-channel attacks that custom or unvetted Python implementations often lack.

B. FastAPI & Uvicorn

These were selected to manage Transport Security. Uvicorn allows for direct integration of SSL certificates (`cert.pem` and `key.pem`), ensuring that all data is wrapped in TLS before it ever hits the network.

C. Bcrypt

Bcrypt was selected for its built-in salt management and tunable adaptive cost factor. This ensures resilience against offline brute-force and hardware-accelerated dictionary attacks by intentionally increasing computational latency, thereby satisfying Secure Local Storage and Abuse Control requirements.

VI. SECURITY ANALYSIS

1. Plaintext Confidentiality

The core guarantee of our E2EE design is that the central server, despite handling all routing and database storage, mathematically cannot deduce message plaintexts. Although the program is successful in hiding message content, E2EE protocols by their very nature disclose contextual data, or metadata. This is achieved through the strict inaccessibility of the symmetric keys. The server would require the 256-bit AES shared secret in order to decrypt any payload, since all communications are encrypted locally using AES-GCM. The AES key is derived via HKDF from an asynchronous X3DH. The server never receives the matching private keys or the new Ephemeral private keys created on the client device, even while it stores the public Identity Keys and One-Time Prekeys. To derive the shared secret purely from the intercepted public keys, the HbC server would have to solve the Elliptic Curve Discrete Logarithm Problem (ECDLP) on Curve25519. This is computationally impossible with the computing technology available today.

2. Exposed Metadata

Even with strong E2EE, the server necessarily learns contextual information required for service operation. To enforce network policies and prevent spam, the server must maintain a plaintext relational database of the social graph, revealing exactly who is communicating with whom. Additionally, the server completely exposes user activity patterns by logging WebSocket connections, API calls, and message timestamps. Finally, the server may still examine precise ciphertext sizes to deduce conversation pace and traffic volume even when a strict 10KB payload limit is enforced to prevent DoS attacks.

3. Limitations and Trade-offs

The primary limitation of our design is its susceptibility to endpoint compromise, as operating system-level malware or keyloggers on a user's device bypass E2EE entirely. Cryptographically, we prioritized system simplicity over strict per-message forward secrecy by utilizing static session keys with incrementing counters, rather than a complex Double Ratchet algorithm. At the session level, this guarantees forward secrecy, but if client memory is extracted, it leaves active sessions exposed. Additionally, we intentionally omitted metadata obfuscation techniques and multi-device synchronization to maintain high-performance, real-time message delivery and minimize the overall cryptographic attack surface.

VII. TESTING & EVALUATION

1. Demonstration for key functions

A. Local State Encryption / Decryption (Secure Storage)

The client protects local state by encrypting the state file before saving it to disk. A user-provided passphrase is used to derive the decryption key, so sensitive data such as private keys, tokens, and local message history are not stored in plaintext. In the demonstration, the encrypted state file was first shown.

```
clientA > {} client_state.json > ...
1  {
2    "salt": "/fiyP14wSj3P1bhSgAsmXw==",
3    "nonce": "Timvfp3CVmz6SQdu",
4    "ciphertext": "uph8MRzgbL/ul25rH0xO8b/YmEoo
5  }
```

Encrypted **client_state.json** showing only protected storage fields instead of readable plaintext client state. (the ciphertext is super long)

Then successfully restored with the correct passphrase.

```
Enter local storage passphrase:
✅ Loaded saved state for user: b
🔄 Syncing identity key and prekeys to server...
✅ Identity key and prekeys successfully synced to server
🔐 Restored session for b
```

Successful decryption and restoration of client state after entering the correct local storage passphrase.

Finally shown to fail decryption when an incorrect passphrase was entered. This confirms that local state is protected at rest.

```
Enter local storage passphrase:
❌ Wrong local storage passphrase.
```

B. X3DH Session Establishment (First Message)

If the two users had not established a shared secret, the first message in the conversation will include the sender's ephemeral public key and the prekey ID used for the conversation.

```
> hi
🔄 First message to cba. Establishing shared secret...
🔐 Long-term shared secret established with cba
```

The receiver will detect if there is a shared secret between the two. The receiver will use the first message that contains the required data to generate the same shared secret for future encryption and decryption.

```
--- Fetched 1 new message(s) ---
🔄 First message from abc. Establishing shared secret...
🔐 Long-term shared secret established with abc
```

C. Message Encryption & Decryption

```
msg_key = hkdf(shared, f"msg_{counter}".encode())
nonce = os.urandom(12)
ct = aesgcm.encrypt(nonce, plaintext.encode('utf-8'), ad)
"cipher": base64.b64encode(nonce + ct).decode('utf-8')
```

Message is encrypted with AES-256-GCM using a unique

per-message key derived from HKDF using shared secret.

```
data = base64.b64decode(cipher_b64)
nonce, ct = data[:12], data[12:]

aesgcm = AESGCM(msg_key)
plaintext = aesgcm.decrypt(nonce, ct, ad).decode('utf-8')
```

Associated Data (AD) is verified on decryption.

D. *Self-Destructing Message*

```
cba: yes
/ttl 3 ttl test
✅ Sent
> ✅ Delivered
a
✅ Sent
> ✅ Delivered
b
✅ Sent
```

```
abc: ttl test
💣 Self-destructed: "ttl test
abc: a
```

```
2026-04-08 23:46:35 You: yes
2026-04-09 00:25:51 abc: a
2026-04-09 00:29:29 abc: b
```

Message appears, then automatically deletes from local storage after the ttl that sender sets. Server also cleans up after TTL expires.

2. *Testing Cases*

Test Case 1: Identity Key Change Detection

- Objective: To verify whether the system can detect an unexpected identity-key change and resist impersonation or key-substitution attacks.
- Threat: An attacker may try to replace a contact's public identity key so that the victim unknowingly trusts a different key.
- Procedure:
  1. Register two users, User A and User B.
  2. Let User A and User B become friends.
  3. Let User A verify User B's fingerprint.
  4. Delete User B's local client state so that User B generates a new identity key after restarting.
  5. Restart User B and log in again.
  6. Let User A attempt to communicate with User B.
- Expected Result: The client should detect that B's identity key has changed. If B was previously verified, the system should warn the user and require re-verification instead of silently accepting the new key.
- Observed Result: The system displayed a key-change warning and blocked continued trust for the previously verified contact until re-verification was performed.
- Security Meaning: This shows that the design can resist simple impersonation or key-substitution attempts by detecting identity-key inconsistency.

```
Choice: 6
Enter contact username to verify: b

--- Fingerprint for b ---
Safety Number: e5625fc689b7d841748861fb7236605c
--------------------------------
Mark as verified? (y/n): y
✅ b has been marked as verified!
```

Fingerprint verification for contact B.

```
> hi
🔄 First message to b. Establishing shared secret...
⚠️ WARNING: Identity key for b has CHANGED!
   Possible key compromise or they reinstalled the app.
   Continuing with warning (you had not verified their previous key).
   Continuing with warning (you previously verified).
🔒 Long-term shared secret established with b
🔑 Safety number / Fingerprint for b: 070263e864d150bb8abf816f43b9134b
   Verify this out-of-band with the contact.
Mark as verified? (y/n):
```

Warning shown after B's identity key changed.

Test Case 2: Replay Attack Resistance

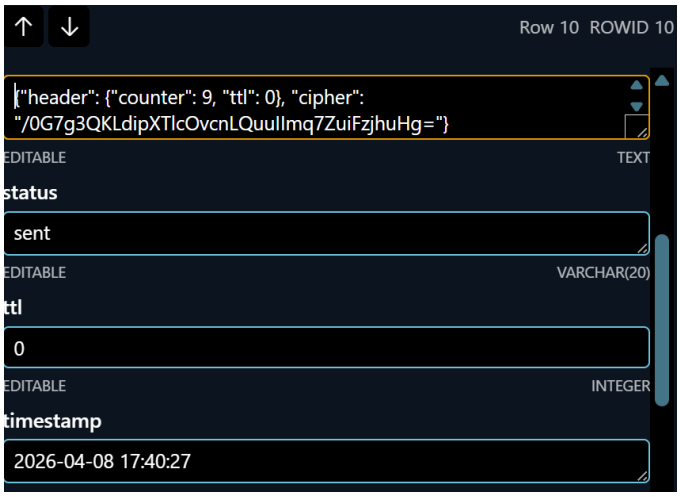
- Objective: To verify the system can resist replay attacks (attacker resending old ciphertexts).
- Threat: An attacker may capture an old ciphertext and attempt to resend it so the recipient treats it as a fresh message.
- Procedure:
  1. User A sends three messages to User B who is currently not connected with WebSocket.
  2. Copy the ciphertext of the first message to second message in database.
  3. Observe User B's message fetching
- Expected Result: The second message should be ignored and silently dropped. User B should only see the first and third messages.
- Observed Result: The replay message did get ignored and not be shown on User B chat.
- Security Meaning: This shows that the design can resist attackers from using the same ciphertext for attack.

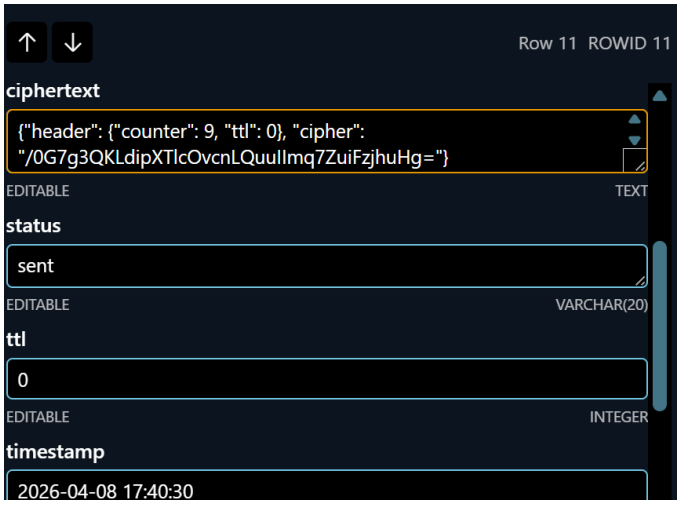
```
a
✅ Sent
> test replay
✅ Sent
> b
✅ Sent
>
```

Messages sent from User A (username "abc").

```
2026-04-08T17:40:27 abc: a
2026-04-08T17:40:31 abc: b
```

Messages received from User B.





Copied ciphertext of the first message to second message in database.

Overall, the testing cases given can achieve the expected results, and the program is able to handle different situations as required.

VIII. FUTURE WORKS

While the current implementation achieves robust end-to-end encryption for 1:1 messaging, there are several avenues for future enhancement to bring the system closer to modern, enterprise-grade secure messaging protocols include:

A. Multi-Device Synchronization

Modifying the protocol to enable multi-device workflows by treating each user's device as an independent cryptographic endpoint with distinct prekey bundles.

B. Group Messaging

Expanding to N-way group chats using standard pairwise encryption would require encrypting and sending N copies of every message, scaling poorly O(N) overhead. In order to achieve efficient O(1) message distribution, future work will implement a scalable Sender Keys protocol, where users generate a random symmetric chain key, encrypt it pairwise for group members once, and then use it to symmetrically encrypt subsequent group messages.

IX. REFERENCES

bcrypt. (2026). *bcrypt* (Version 5.0.0) [Python library]. https://pypi.org/project/bcrypt/

Cryptography.io. (n.d.). *AESGCM — Authenticated encryption*. In *Cryptography 42.0.0 documentation.* Retrieved April 7, 2026, from https://cryptography.io/en/latest/hazmat/primitives/aead/#cryptography.hazmat.primitives.ciphers.aead.AESGCM

Encode OSS. (2026). *FastAPI* [Web framework]. https://fastapi.tiangolo.com/

SQLAlchemy. (2026). SQLAlchemy (Version 2.0+) [Python library]. https://www.sqlalchemy.org/

SSOJet. (n.d.). *Generate keypair using X25519 in Python*. Retrieved April 7, 2026, from https://ssojet.com/keypair-generation/generate-keypair-using-x25519-in-python#signing-data-with-the-private-key

The Python Software Foundation. (2026). *Python 3.14.4 Standard Library*. https://docs.python.org/3/library/

Uvicorn. (2026). *uvicorn* (Version 0.44.0) [Python library]. https://www.uvicorn.org/

Websocket-client. (2026). websocket-client (Version 1.9) [Python library]. https://pypi.org/project/websocket-client/